Tritorc Tender Scanner — Judge's Runbook & Roadmap

Nov 7, 2026 · @an

Your zip asks for more than the brief says: the sample Excel has 24 columns, not 4, and your three real GeM tenders hold traps that break a naive scanner. This plan builds to the sample file and targets full marks in all six rubric areas.

What is really in your zip

File	What it is	What it means for you
Tender_Keyword_Scanner_Assignment.pdf	The brief: React + Node/Express, PDF/DOCX in, keyword scan, Excel out. 100 marks, 4–6 hours	Five things to submit: repo, README, AI_NOTES.txt, DEPLOY.md, screen recording
keywords.csv	20 keywords in one column headed `keyword`	Load it as config at startup, not inline, and skip the header row
sample_tender_scan_report.xlsx	One sheet, 3 fictional rows, **24 columns**, a disclaimer in row 6	The real output spec. The brief only demands 4 columns
GeM-Bidding-9563430.pdf	6-page OPGC (Odisha) bid for a tube beveling machine	Your "clearly not related" case: 0 keyword hits
GeM-Bidding-9605104.pdf	8-page BHEL bid for 9 hydraulic nuts	Your borderline case: 0 hits on the default list, yet close to bolt tensioning

File	What it is	What it means for you
GeM-Bidding-9649647.pdf	11-page SAIL Rourkela bid for a hydraulic torque wrench with power pack	Your "clearly related" case: 6 + 15 hits

No DOCX file is included, so you must make two of your own to exercise `mammoth`: one relevant, one not.

Traps found by running your real files

I ran `pdf-parse` 2.4.5 on the three PDFs and read every cell of the sample Excel. Each trap below is something a reviewer can catch.

Trap	Evidence in your files	Fix
Output has 24 columns, not 4	Sample header runs Document Name → Relevance to Tritorc; the brief says "at minimum" 4	Produce all 24 in the sample's order, with the sample's header and verdict colours
Verdict wording conflicts	Brief says Yes / Possible / No; sample says Related / Possible / Not Related	Follow the sample in the Excel; state the mapping (Related = Yes) in the README
Match Count means distinct keywords	Sample row 1 lists 6 keywords and shows 6; zero matches prints `(none found)`	Count distinct keywords, not occurrences
Distinct-count thresholds mislabel your best tender	SAIL buys a hydraulic torque wrench yet hits only 2 distinct keywords, so "1–2 = Possible" under-rates it	Add an occurrence rule (next sections)
`pdf-parse` changed its API	`npm i pdf-parse` now gives 2.x: `new PDFParse({ data }).getText()`. Old tutorial code `pdfParse(buffer)` fails	Pin the version and hide it behind one extractor module

Trap	Evidence in your files	Fix
Hindi text extracts as garbage	Every label is bilingual; the Hindi half comes out as duplicated fragments with control characters. The English half is clean, e.g. `Bid Number: GEM/2026/B/7824594`	Anchor every field regex on the English label only
A loose regex returns boilerplate as a value	BHEL's PDF has no Estimated Bid Value row, but a terms clause starts "Estimated Bid Value indicated above is being declared…"	Require a number right after the label and a tab, else `Not Found`
Raw substring stemming creates false hits	All three PDFs contain "extension", which contains "tension"	Match whole words on stemmed tokens, never raw substrings
Many sample columns have no source in GeM bids	Tender ID, Document Cost, Tender Fee, Website, Corrigendum have no value in any of the three PDFs. Estimated Bid Value exists only in OPGC's (721570). All three say EMD Required: No	Output `Not Found` as the sample's own note asks; output `Not Required` for EMD

How a judge will score you

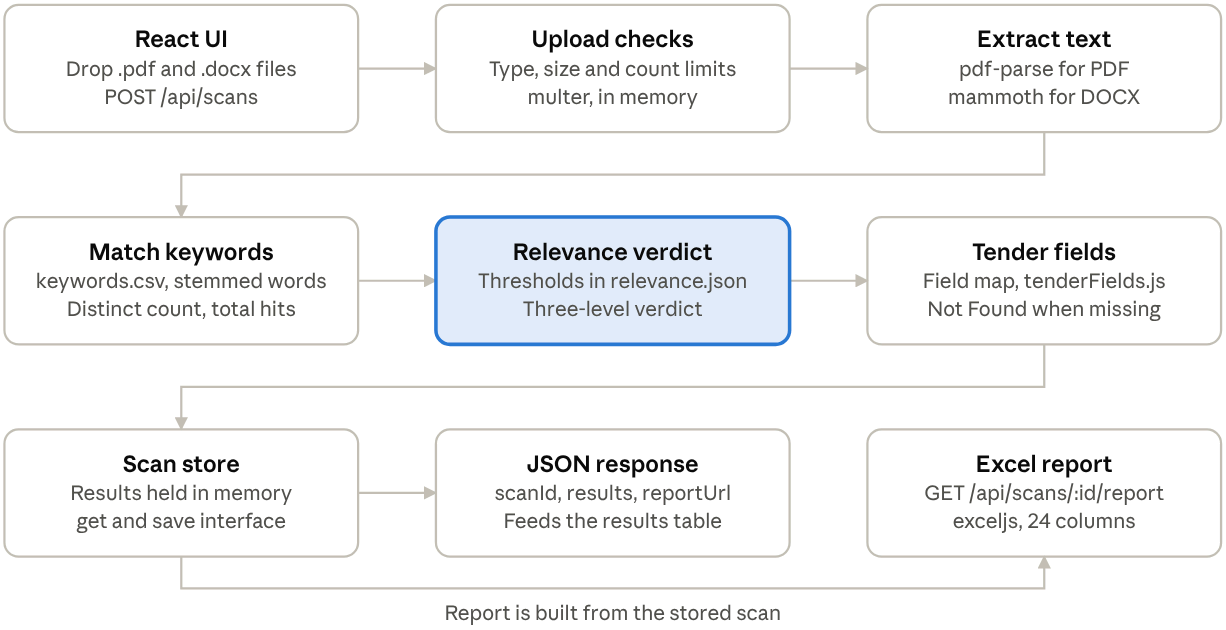
65 of the 100 marks sit in server logic that can be tested without any UI, so build and test the backend first and the interface last.

Area	Marks	Full marks looks like	Where interns lose it
Backend: upload, extraction, search	25	Multi-file upload with type and size limits. PDF and DOCX both work. Matching ignores case and handles plurals. One bad file never fails the batch	Only PDF works. A scanned or corrupt file crashes the request. Substring false hits

Area	Marks	Full marks looks like	Where interns lose it
Excel report	20	All columns present, values equal the scan, opens cleanly in Excel, downloads from the UI with a sensible filename	Wrong Match Count. No `Not Found` fallback. Download link works only on localhost
Relevance logic	20	Verdict computed from thresholds held in config, unit-tested at 0, 1, 2, 3+ and on the SAIL tender	Verdict hardcoded or thresholds buried inside a controller
Frontend and integration	20	Drag-and-drop with file list, loading and error states, results table that mirrors the Excel, download button, usable at 375 px width	No loading state. Table overflows on mobile. Failed uploads vanish silently
Code quality and structure	10	Routes, controllers, services separated. Config out of code. Consistent naming. Tests. No dead code. Incremental git history	One 400-line file. Commented-out code. A single "initial commit"
AI usage and documentation	5	AI_NOTES.txt with 2–3 real prompts and an honest used / modified note. README that runs first try. Specific DEPLOY.md	Generic notes. Setup steps that do not work

Architecture and folder structure

One upload becomes a verdict per file, then an Excel report



scan pipeline · 9 steps in 3 stages

The highlighted step is the 20-mark relevance logic, so keep its thresholds in config and never in a controller. The folder layout that implements the pipeline:

```
tender-scanner/
  client/src/
    api/            scanApi.js
    components/     UploadDropzone, FileList, ResultsTable, RelevanceBadge,
                    DownloadReportButton, StatusBanner
    hooks/          useScan.js
    pages/          ScannerPage.jsx
    styles/         _variables.scss, main.scss
  server/
    data/           keywords.csv
    fixtures/       3 GeM PDFs, 2 DOCX files
    src/
      config/       env.js, keywords.js, relevance.js, relevance.json,
                    tenderFields.js, reportColumns.js
      routes/       scanRoutes.js
      controllers/  scanController.js
      middleware/   uploadFiles.js, errorHandler.js
```

```
      services/    extraction/, matching/, relevance/, tender/, report/,
scan/
      utils/       AppError.js, logger.js
    tests/
  docs/            reference/ (brief, sample Excel), screenshots/
  README.md  AI_NOTES.txt  DEPLOY.md
```

Matching and relevance logic to implement

Match on stemmed whole words, count distinct keywords for the Match Count column, and judge relevance from distinct keywords plus total hits, with every threshold in a config file.

Matching steps

1. Normalise the text: lowercase, Unicode NFKC, hyphens and underscores to spaces, collapse whitespace. This makes "Pre-tensioning" and "pre tensioning" equal.
2. Split into word tokens and stem each one with a small library stemmer, so "wrenches" becomes "wrench" and "tensioned" becomes "tension".
3. Stem each keyword the same way and find it as an ordered token sequence. A `maxWordGap` of 1 lets "torque wrench calibration" count as "Torque calibration", which is how the sample behaves.
4. Record the hit count per keyword. A keyword is matched when its count is above zero.
5. Print matched keywords in the CSV's spelling, ordered by first appearance in the document, and say so in the README.

Verdict rule (file `server/src/config/relevance.json`)

```
{
  "relatedMinDistinct": 3,
  "relatedMinTotalHits": 10,
  "possibleMinDistinct": 1,
  "maxWordGap": 1
}
```

Related when distinct keywords ≥ 3, or when at least 1 keyword matched and total hits ≥ 10. Possible when at least 1 keyword matched. Otherwise Not Related. The hits rule is the one that rescues genuinely relevant tenders that mention one or two terms many times.

Worked example on your three tenders (counts from my run on the PDF text)

Tender	Distinct, gap 0	Distinct, gap 1	Total hits	Verdict
OPGC tube beveling machine	0	0	0	Not Related
BHEL hydraulic nuts	0	0	0	Not Related
SAIL hydraulic torque wrench	2	3	21–22	Related under either setting

Your "Possible" demo: BHEL scores Not Related on the default list. Add the term `hydraulic nut` through the editable keyword list and it matches about 8 times, giving 1 distinct keyword and Possible. That shows the list is genuinely configurable and gives you the borderline case the brief asks for. Whether hydraulic nuts belong in Tritorc's list is a business call, so present it as a demonstration, not a fact.

Roadmap

About 6.5 hours in total: 5 hours for the brief's core, plus 75 minutes for the tender-field extraction that lifts your Excel from the brief's bare minimum to the sample's 24 columns. Commit once at every gate so the git history shows the work growing.

Phase	Time box	Done when (gate)
0. Setup and conventions	20 min	Repo on GitHub, client and server both boot, lint passes, first commit pushed
1. Keyword config and text extraction	40 min	Your three PDFs and two DOCX files return non-empty text through one `extractText` function. A corrupt file returns a typed error, not a crash
2. Matching and relevance	50 min	Unit tests pass at 0, 1, 2 and 3+ distinct keywords, and the SAIL tender is Related
3. Tender field extraction	75 min	Each PDF yields the right value or `Not Found` for every field in the traps table

Phase	Time box	Done when (gate)
4. Excel report	40 min	File opens in Excel with the sample's header colour, frozen first row and 24 columns. Cell values equal the API result
5. REST API	35 min	One `curl` uploads three files and a second `curl` downloads the .xlsx
6. React UI	70 min	Upload, results and download work at 375 px and 1280 px wide
7. Tests and fixtures	25 min	`npm test` is green and a `fixtures/` folder holds the PDFs and DOCX files
8. Docs and evidence	25 min	README, AI_NOTES.txt, DEPLOY.md and a screen recording are done
9. Self-judge pass	15 min	Every box in the final checklist is ticked

Master prompt

Paste this first in every AI session, whichever assistant you use. It carries the stack, the output spec and the style rules, so each phase prompt below stays short. Re-paste it if a long session starts drifting.

```
You are my pair-programmer on a take-home project: a Tender / Scope-of-Work
Keyword Scanner for Tritorc. I give you one small task at a time.

PRODUCT
Users upload .pdf or .docx tender files. The backend extracts text, matches
keywords from keywords.csv, computes a relevance verdict, and returns JSON
plus a downloadable .xlsx report.

STACK
React (Vite) with SCSS. Node 22 + Express, ES modules. multer, pdf-parse
2.4.5 (pinned, class API: new PDFParse({ data }).getText()), mammoth,
exceljs, the `stemmer` package, vitest + supertest.

OUTPUT SPEC
docs/reference/sample_tender_scan_report.xlsx defines the report. Sheet
"Tender Scan Report", these 24 columns in this order: Document Name, Tender
```

```
ID, Tender No, Tender Authority, Location, Opening Date, Closing Date, Tender
Amount, EMD, Document Cost, Tender Fee, Tab Name, Technical Qualification,
Financial Qualification, Scope of Work, Tender Summary, Tender Description,
Corrigendum, Purchaser Address, Email, Website, Matched Keywords, Match
Count, Relevance to Tritorc.
Missing values print "Not Found". Zero matches print "(none found)". Match
Count is the number of DISTINCT keywords matched. Verdicts are Related /
Possible / Not Related.

ARCHITECTURE RULES
- server/src is layered: routes -> controllers -> services. Services are pure
functions with no Express imports.
- Config lives in server/src/config. Keywords load from
server/data/keywords.csv. Relevance thresholds load from relevance.json. No
magic numbers inline.
- One responsibility per file. Files under about 120 lines, functions under
about 25.

CODE STYLE RULES
- Descriptive domain names (tender, scan, verdict, keywordHit). No tmp,
data2, res1.
- Comment only the WHY, never what the code does. JSDoc only on exported
service functions.
- No console.log; use the logger module. async/await only. One error-handling
middleware with an AppError class carrying a status code.
- No dead code, no TODOs, no placeholder text, no emoji.
- Validate at the boundary: extension and MIME type, size limit from env.
- Do not add features or dependencies I did not ask for. If you propose a
dependency, justify it in one line.

HOW TO ANSWER
- Show only the files you create or change, each with its path.
- After the code, list in at most 3 bullets what you deliberately did not do.
- If a requirement is ambiguous, ask me one question before writing code.
- When I paste an error or failing test, fix the cause, not the symptom.
```

Runbook: phases 0–2

Each phase is one prompt, one check, one commit. Run the master prompt first, then the phase prompt.

Phase 0: setup and conventions (20 min)

```
Task: scaffold the repo. Create exactly this structure, adding a file only
where it has real content:

tender-scanner/
  client/src/{api,components,hooks,pages,styles}
  server/
    data/keywords.csv
    src/{config,routes,controllers,services,middleware,utils}
    tests/  fixtures/
  docs/{reference,screenshots}
  README.md  AI_NOTES.txt  DEPLOY.md

Server: app.js builds the Express app (cors limited to CLIENT_ORIGIN, json,
routes, error middleware). server.js only calls listen. config/env.js reads
PORT, CLIENT_ORIGIN, MAX_UPLOAD_MB and throws a clear error if one is
missing. utils/logger.js is a thin wrapper over console.
middleware/errorHandler.js plus utils/AppError.js. One route: GET
/api/health.
Client: Vite + React + SCSS, dev proxy of /api to the server, empty App
shell.
Both apps: ESLint, Prettier, .editorconfig, .env.example, .gitignore. No
business logic yet.
```

Check: both apps start, `GET /api/health` returns 200, `npm run lint` is clean. Copy the zip's `keywords.csv` to `server/data/` and the brief and sample Excel to `docs/reference/`.

Commit: `chore: scaffold client and server`

Phase 1: keyword config and text extraction (40 min)

First write two DOCX fixtures yourself in Word, `relevant_sow.docx` (mentions controlled bolting, flange management, hydraulic torque wrench) and `civil_works.docx` (piling, concrete, drainage). Writing them yourself keeps the content yours. Put them and the three GeM PDFs in `server/fixtures/`.

```
Task: keyword config and text extraction.
1. config/keywords.js: loadKeywords() reads server/data/keywords.csv (header
column is `keyword`), trims, drops blanks and case-insensitive duplicates,
caches the result, throws AppError 500 if the file is missing.
2. services/extraction/pdfExtractor.js using the pdf-parse 2.x class API;
always call destroy() in a finally block.
services/extraction/docxExtractor.js using mammoth.extractRawText.
services/extraction/index.js exports extractText({ buffer, originalName })
and picks the extractor by file extension.
3. Errors: unsupported extension -> AppError 415. Parser failure -> AppError
422 "Could not read <name>". Fewer than 20 characters of text -> AppError 422
"No readable text (scanned PDF?)". No OCR.
Write vitest tests against server/fixtures: each fixture returns non-empty
text, a renamed .txt file is rejected, a truncated PDF buffer is rejected.
```

Check: the SAIL PDF text contains `HYDRAULIC TORQUE WRENCH WITH POWER PACK`. Hindi fragments in the text are expected and harmless.

Commit: `feat(server): load keywords and extract text from PDF and DOCX`

Phase 2: matching engine and relevance (50 min)

```
Task: matching engine and relevance verdict. No Express code.
Files: services/matching/normalizeText.js, tokenizeWords.js,
findKeywordHits.js, scanText.js; services/relevance/classifyRelevance.js;
config/relevance.js loading server/src/config/relevance.json
{relatedMinDistinct:3, relatedMinTotalHits:10, possibleMinDistinct:1,
maxWordGap:1}.
Rules:
- Normalise: lowercase, NFKC, hyphens and underscores to spaces, collapse
whitespace.
- Tokenise to words, stem with the `stemmer` package, stem the keyword the
same way, match as an ordered token sequence allowing up to maxWordGap extra
words between keyword words.
- scanText(text, keywords) returns { matches: [{ keyword, hits, firstIndex
}], distinctCount, totalHits }, matches ordered by firstIndex.
- classifyRelevance(scan, thresholds) returns "Related", "Possible" or "Not
Related": Related if distinctCount >= relatedMinDistinct, or (distinctCount
>= possibleMinDistinct and totalHits >= relatedMinTotalHits); Possible if
distinctCount >= possibleMinDistinct; else Not Related.
Tests (table-driven): "Torque wrenches" matches "Torque wrench"; "Pre-
tensioning" matches "Pre-tensioning"; the word "extension" must NOT match any
tension keyword; "torque wrench calibration" matches "Torque calibration"
only when maxWordGap >= 1; verdict boundaries at distinct 0, 1, 2, 3 and at
distinct 1 with 10 hits.
```

Check: add one integration test over the three GeM PDFs expecting Not Related, Not Related, Related. Expect the SAIL tender to show roughly 2–3 distinct keywords and 21–22 hits. If your stemmer disagrees, inspect the tokens before touching the test.

Commit: `feat(server): keyword matching with stemming and relevance verdicts`

Runbook: phases 3–4

Phase 3: tender field extraction (75 min)

The sample's 24 columns come from the tender itself, so the extractor needs a field map. The mapping below is my judgement of how GeM labels line up with the sample's columns; write it into the README as a design decision.

Excel column	Source in the GeM PDF (English label only)	Rule
Tender No	`Bid Number`	As printed, e.g. GEM/2026/B/7824594
Tender Authority	`Organisation Name`	As printed
Location	First consignee address	Text after the last comma, or after the pincode if there is no comma
Opening Date, Closing Date	`Bid Opening Date/Time`, `Bid End Date/Time`	Date part only, written DD-Mon-YYYY like the sample. The opening label wraps onto two lines
Tender Amount	`Estimated Bid Value`	Digits must follow the label and a tab. Write `INR 721,570` (assumes rupees, GeM being an Indian portal)
EMD	`EMD Detail` then `Required`	`Not Required` when No; `Required (see document)` when Yes
Tab Name	`Relevant Categories selected for notification`	As printed
Tender Description	`Item Category`	Full text
Scope of Work	Text under `REQUIREMENT DESCRIPTION->`	Falls back to Item Category
Technical Qualification	Text under `ELIGIBILITY CRITERIA->`	Whitespace collapsed, else Not Found
Tender Summary	Built from Item Category, Total Quantity, Type of Bid	A fixed template sentence, not invented prose
Purchaser Address	Office Name, Organisation Name, consignee address	Joined with commas
Email	`Buyer Email id`	As printed

Excel column	Source in the GeM PDF (English label only)	Rule
Tender ID, Document Cost, Tender Fee, Corrigendum, Website, Financial Qualification	No value in these bids	`Not Found`

```
Task: tender field extraction for GeM bid PDFs.
Create services/tender/extractTenderFields.js plus a declarative
config/tenderFields.js. Each definition is { column, pattern, transform }. A
small readField(text, definition) applies it and returns "Not Found" when the
pattern misses or the transform rejects the value. Do not write one if-
statement per field.
Rules: match on the English label only and ignore Hindi text. A label can
wrap onto the next line ("Bid Opening" then "Date/Time"), and the value
follows after a tab or spaces. Estimated Bid Value must be followed by
digits, because a terms clause also begins with the same words. Dates become
DD-Mon-YYYY. The column list and order live in config/reportColumns.js,
shared with the Excel builder.
Here is the mapping table: <paste the table above>.
Write tests with these golden values for the three fixtures.
```

Golden values to assert, read from your three PDFs:

Field	OPGC (9563430)	BHEL (9605104)	SAIL (9649647)
Tender No	GEM/2026/B/7748755	GEM/2026/B/7785415	GEM/2026/B/7824594
Tender Authority	Odisha Power Generation Corporation Limited	Bharat Heavy Electricals Limited (bhel)	Rourkela Steel Plant
Opening Date	22-Jul-2026	27-Jul-2026	03-Aug-2026
Closing Date	22-Jul-2026	27-Jul-2026	03-Aug-2026
Tender Amount	INR 721,570	Not Found	Not Found
EMD	Not Required	Not Required	Not Required
Email	mohit.murmu@opgc.co.in	ksantosh@bhel.in	buyer26.rsp.or@gembuyer.in

Check: if a value differs, print the raw `pdf-parse` text around the label before changing the pattern. Anything in a column marked Not Found above must stay `Not Found`, never boilerplate.

Commit: `feat(server): extract tender fields from GeM bids with a declarative field map`

Phase 4: Excel report (40 min)

```
Task: Excel report with exceljs. services/report/buildWorkbook(results)
returns a Buffer.
- Sheet "Tender Scan Report", header row in the order of
config/reportColumns.js, bold white text on fill 1E3A5F, freeze panes at B2,
wrap text, vertical top.
- Column widths A..X: 30, 15, 16, 24, 20, 13, 13, 18, 16, 14, 13, 15, 30, 28,
34, 34, 36, 30, 30, 24, 20, 40, 11, 16.
- Matched Keywords joined with ", ", or "(none found)". Match Count is a
number. Relevance cell fill: Related D9F2D9, Possible FDEBD3, Not Related
F2F2F2.
- Second sheet "Keyword Breakdown" with Document, Keyword, Hits for every
match.
- Do not copy the sample's fictional-data disclaimer row.
Test: load docs/reference/sample_tender_scan_report.xlsx, read its header
row, and assert our generated header row is identical. Add a test that a
result with no matches renders "(none found)" and 0.
```

Check: open your file next to the sample in Excel. Headers, colours, frozen row and widths should look alike.

Commit: `feat(server): generate Excel report matching the sample layout`

Runbook: phases 5–6

Phase 5: REST API (35 min)

The contract the frontend will depend on:

Method and path	Purpose	Success response
`GET /api/health`	Liveness check	`200 { status: "ok" }`

Method and path	Purpose	Success response
`GET /api/keywords`	The active keyword list	`200 { keywords: [...] }`
`POST /api/scans`	Multipart field `files`, up to 10 files	`201 { scanId, scannedAt, results: [...], reportUrl }`
`GET /api/scans/:id/report`	Download the Excel file	`200` .xlsx with `Content-Disposition: attachment`

```
Task: REST API for scans.
- routes/scanRoutes.js, controllers/scanController.js (thin: parse request, call a service, send response), services/scan/scanDocuments.js (orchestrates extractText, scanText, classifyRelevance, extractTenderFields for each file with Promise.allSettled), services/scan/scanStore.js (in-memory Map with a one-hour expiry behind save(id, scan) and get(id), so MongoDB could replace it later).
- middleware/uploadFiles.js: multer memoryStorage, field "files", max 10 files, per-file limit from MAX_UPLOAD_MB, extension filter .pdf and .docx. Map multer errors to AppError 400 or 413 in the error middleware.
- A file that fails to read must not fail the batch: its result is { documentName, status: "failed", error } and its Excel row shows "Not Assessed" in Relevance with the error text in Matched Keywords. Successful files carry status "scanned".
- No files uploaded -> 400. Unknown scan id -> 404.
Tests with supertest: three GeM PDFs give 201 and verdicts Not Related, Not Related, Related; a .txt file is rejected; a batch with one corrupt PDF still returns 201 with one failed entry; the report endpoint returns the xlsx content type.
```

Check: from the terminal, `curl -F "files=@server/fixtures/GeM-Bidding-9649647.pdf" http://localhost:4000/api/scans`, then download the report from the returned URL and open it.

Commit: `feat(server): scan and report endpoints with per-file error handling`

Phase 6: React UI (70 min)

```
Task: React frontend, mobile first, SCSS, no UI component library.
Structure: api/scanApi.js (fetch wrappers; base URL from
import.meta.env.VITE_API_URL), hooks/useScan.js (state: idle, uploading,
done, error), components/UploadDropzone.jsx, FileList.jsx, ResultsTable.jsx,
RelevanceBadge.jsx, DownloadReportButton.jsx, StatusBanner.jsx,
pages/ScannerPage.jsx, styles/_variables.scss.
Behaviour:
- Dropzone supports drag and drop and a browse button, accepts only .pdf and
.docx, shows each chosen file with a remove button, and rejects other types
with a visible message.
- Scan button is disabled with no files and while uploading. Show a progress
or busy state. Show server errors in StatusBanner with role="alert".
- ResultsTable shows Document Name, Matched Keywords (as chips), Match Count
and a RelevanceBadge, the same summary as the Excel. A row expands to show
the extracted tender fields. Failed files show their error text.
- DownloadReportButton downloads from reportUrl and keeps the server's
filename.
Responsive: under 640px the table becomes stacked cards; touch targets at
least 44px; visible focus outlines; status text in an aria-live region.
Style: reuse the Excel palette as variables: navy 1E3A5F, Related D9F2D9,
Possible FDEBD3, Not Related F2F2F2.
```

Check: test at 375 px and 1280 px in browser dev tools. Stop the server to see the error state. Upload a .txt file to see the rejection. Upload the three PDFs and expect Not Related, Not Related, Related, then download and open the report.

Commit: `feat(client): upload, results table and report download`

Runbook: phases 7–8

Phase 7: tests and fixtures (25 min)

```
Task: audit the test suite. For each area, tell me whether a test exists and
what it asserts: extraction errors, normalisation and stemming, relevance
boundaries, tender field golden values, workbook header against the sample,
API happy path and failure paths. Add only the missing tests, no duplicates.
Add a root `npm test` script that runs the server suite. Do not mock the
PDFs: use server/fixtures.
```

Check: `npm test` is green from a fresh clone. The fixtures folder holds the three GeM PDFs and your two DOCX files. If time remains, the only worthwhile bonus is MongoDB scan history behind the `scanStore` interface; skip it if the core is not yet polished.

Commit: `test: fill coverage gaps and document fixtures`

Phase 8: the four submission files (25 min)

```
Task: draft README.md for this repository. Read package.json scripts,
.env.example files and the routes first. Sections: what it does, stack,
prerequisites (Node version), setup for server and client with exact
commands, running tests, API table, how to edit keywords and relevance
thresholds, design decisions (field mapping, Match Count meaning, verdict
wording mapping Related = Yes, gap tolerance, keyword ordering), known
limitations (no OCR for scanned PDFs, English labels only, in-memory scan
store), project structure. Every command must be runnable as written. Do not
describe any feature that is not in the code.
```

Then follow your own README in a fresh clone. Whatever fails is a README bug.

File	What it must contain
README.md	Setup for frontend and backend, as above
AI_NOTES.txt	2–3 real prompts, what came back, and for each one "used directly" or "modified" with the change and the reason
DEPLOY.md	Frontend on a static host such as Vercel or Netlify, with `VITE_API_URL` set at build time. Backend on a Node host such as Render or Railway, with `PORT`, `CLIENT_ORIGIN`, `MAX_UPLOAD_MB` set in the host's dashboard and never committed. A concrete production API URL, e.g. `https://tender-scanner-api.example.com/api`. A note that the in-memory scan store resets on restart, so a real deployment would use MongoDB

File	What it must contain
Evidence	A 60–90 second screen recording: upload the three PDFs, scan, read results, download, open the Excel. Add 4 screenshots in `docs/screenshots`: desktop results, 375 px results, an error state, the opened Excel

AI_NOTES.txt template. Replace every angle-bracket item with what actually happened.

```
AI_NOTES
Tool used: <name and version>

Prompt 1: matching engine (phase 2)
Prompt: <the prompt as sent>
What came back: <one or two lines>
Used: <directly | modified>
What I changed and why: <the real change, or "nothing">

Prompt 2: tender field extraction (phase 3)
...
Prompt 3: React UI (phase 6)
...
```

Pick at least one prompt where the output needed fixing, because an honest correction reads better than three "used directly". Do not invent one that did not happen.

Commit: `docs: add README, AI notes and deployment guide`

Making it read as your own clean code

Code reads as yours when it is small, consistent and explainable. Generated code usually gives itself away in the ways below, so check each one before you submit.

Tell-tale	Why a judge notices	What to do instead
One large `index.js` or `App.jsx`	No separation of concerns, hard to review	Routes, controllers, services. Files under about 120 lines

Tell-tale	Why a judge notices	What to do instead
A comment on every line, such as "initialise express app"	Comments restate the code	Comment only the why, for example why Hindi text is ignored
Names like `data`, `result`, `temp`, `handleStuff`	Nothing says what the value means	Domain names: `tender`, `scan`, `verdict`, `keywordHit`
Keywords and thresholds inline	Fails the "not hardcoded" criterion	`keywords.csv` and `relevance.json`
A single "initial commit"	No visible progress	One commit per phase gate with a conventional message
Unused packages, `console.log`, commented-out code	Looks pasted	Run `npx depcheck`, enable ESLint `no-unused-vars`, search for `console.log`
No tests, or tests that mock everything	Nothing proves it works	Real fixtures: your three PDFs and two DOCX files
Extra features nobody asked for	Looks like padding	Build the rubric first; one bonus at most
A README that has never been run	First setup step fails	Follow it in a fresh clone

You will likely be asked about your code, so own every file. Read what the AI wrote, and be able to rewrite the core of each service from memory. Be ready for these:

1. Why count distinct keywords for Match Count, and why add a total-hits rule?
2. How does "wrenches" match "wrench", and why not use plain substring search?
3. Why does the Hindi text not break your field extraction?
4. What happens when a PDF is a scan with no text layer?
5. Where do the keywords and thresholds live, and how would the sales team change them?
6. What happens when one file in a batch of five is corrupt?
7. Why is the scan store in memory, and what changes if you move it to MongoDB?
8. Which environment variables does production need, and where are they set?

Final self-judge checklist

Tick every box in a fresh clone before you send the repo link.

- [ ] Default 20 keywords load from `keywords.csv` with no code change
- [ ] PDF and DOCX both extract; a corrupt or text-less file gives a clear error and does not fail the batch
- [ ] "Torque wrenches" matches "Torque wrench"; "extension" never matches a tension keyword
- [ ] The three GeM tenders score Not Related, Not Related, Related
- [ ] The verdict comes from `relevance.json`; changing a threshold changes the result with no code edit
- [ ] The Excel has the sample's 24 columns in order, header colour 1E3A5F, frozen first row, `Not Found` and `(none found)` where due
- [ ] Excel values equal what the results table shows
- [ ] The report downloads from the UI and opens in Excel without a repair prompt
- [ ] The UI works at 375 px and 1280 px, with loading and error states
- [ ] `npm test` is green; no `console.log`, dead code or unused packages
- [ ] Git history has about ten meaningful commits
- [ ] README works from a fresh clone; AI_NOTES.txt is truthful; DEPLOY.md names real hosts and variables
- [ ] Screen recording and screenshots are in the repo or linked from the README
- [ ] You can answer the eight questions above without opening the code